

Explainable IDS Pipeline: Execution Report and Documentation

April 21, 2026

1 Overview

This document summarizes the purpose, workflow, fixes, and verified execution results of `pipeline.py`. The script implements an explainable intrusion detection system (IDS) workflow on the NSL-KDD dataset. It performs data loading, preprocessing, model training, evaluation, explanation generation, stability analysis, and adversarial analysis in a single end-to-end pipeline.

2 Pipeline Scope

The pipeline executes the following stages:

1. Load the NSL-KDD training and test sets.
2. Convert the task to binary classification: `normal` vs. `attack`.
3. One-hot encode categorical features and scale inputs with MinMax normalization.
4. Train three classifiers:
 - Logistic Regression
 - Random Forest
 - Multi-Layer Perceptron (MLP)
5. Evaluate each model using classification metrics and PR-AUC.
6. Generate explanations with SHAP and LIME.
7. Measure explanation stability across similar attack samples.
8. Run a SHAP-guided adversarial evasion analysis and a simple defense evaluation.

3 Issues Found and Corrected

The original runtime path had several execution problems. These were corrected directly in `pipeline.py`.

3.1 Dataset Download Failure

The original loader attempted to download the dataset exclusively from `raw.githubusercontent.com`. On this machine, DNS policy returned `NXDOMAIN` for that host, so the pipeline could not start from a clean workspace.

Fix:

- Kept the standard raw GitHub download path as the first attempt.
- Added an automatic fallback using the GitHub API blob endpoint.
- Added a clear, user-facing error if both download paths fail.

3.2 Matplotlib Configuration Warning

Matplotlib tried to write its configuration cache under a non-writable default path, producing warnings at startup.

Fix:

- Set `MPLCONFIGDIR` to a writable local directory under `outputs/.matplotlib`.
- Ensured the backend is configured before importing `pyplot`.

3.3 Label Normalization Robustness

The NSL-KDD family of files can appear with small formatting variations such as trailing periods in labels.

Fix:

- Normalized label strings with whitespace stripping.
- Treated both `normal` and `normal.` as the benign class.

3.4 Explanation and Analysis Edge Cases

Modern SHAP versions can return outputs in different binary-classification formats, and some downstream analysis steps can fail when no valid samples or pairs are available.

Fix:

- Added SHAP output normalization for list-based and ndarray-based return types.
- Added guards for empty attack subsets and empty similarity-pair results.
- Improved final summary handling when adversarial results are unavailable.

4 Verified Execution

The pipeline was run end-to-end successfully after the fixes. During execution, the missing dataset files were downloaded automatically through the GitHub API fallback.

4.1 Dataset Sizes

- KDDTrain+.txt: 19,109,424 bytes
- KDDTest+.txt: 3,441,513 bytes

4.2 Observed Dataset Shapes

- Training rows: 125,973
- Test rows: 22,544
- Final feature count after preprocessing: 122

5 Model Results

The final verified model metrics are shown in Table 1.

Model	Macro F1	PR-AUC
Logistic Regression (baseline)	0.7535	0.8864
Random Forest	0.7726	0.9650
MLP	0.8060	0.9461

Table 1: Verified evaluation results from the final pipeline run.

The MLP achieved the highest macro F1 score, while the Random Forest achieved the best PR-AUC.

6 Explainability Results

6.1 SHAP

The Random Forest SHAP analysis identified the following top features:

- flag_SF
- logged_in
- dst_host_srv_count
- dst_host_same_srv_rate
- same_srv_rate
- diff_srv_rate
- service_private
- dst_host_rerror_rate
- count
- dst_host_srv_serror_rate

The Spearman rank correlation between Random Forest SHAP importances and MLP SHAP importances was:

$$\rho = 0.745 \quad (p = 0.0000)$$

This indicates substantial agreement between the two explanation profiles.

6.2 LIME

The top global LIME features from the evaluated attack instances were:

- service_private
- logged_in
- wrong_fragment
- dst_host_rerror_rate
- rerror_rate
- flag_SF
- service_ecr_i
- dst_host_srv_serror_rate
- protocol_type_icmp
- flag_S0

7 Stability and Adversarial Analysis

7.1 Explanation Stability

For similar attack-sample pairs, the mean Jaccard similarity of the top-10 SHAP features was:

$$0.994 \pm 0.037$$

This indicates very high stability for the selected explanation subset under the implemented similarity criterion.

7.2 SHAP-Guided Attack Results

Table 2 summarizes the adversarial evasion rates.

Perturbed Features	$\epsilon = 0.05$	$\epsilon = 0.10$	$\epsilon = 0.20$
Top-5	34.86%	33.60%	32.26%
Top-10	35.49%	35.04%	34.59%
Top-15	35.49%	34.14%	33.51%

Table 2: Verified evasion rates from the SHAP-guided perturbation analysis.

7.3 Defense Evaluation

The implemented feature-randomization defense did not materially reduce evasion in the observed run:

- Evasion rate without defense: 35.04%
- Evasion rate with defense: 35.04%
- Observed reduction: 0.0%

8 Generated Artifacts

The pipeline produced the following output figures in the **outputs/** directory:

- `shap_beeswarm_rf.png`
- `shap_comparison.png`
- `shap_stability.png`
- `evasion_heatmap.png`

9 How to Run

From the project directory:

```
./venv/bin/python pipeline.py
```

If the dataset files are not already present, the script will attempt to download:

- `KDDTrain+.txt`
- `KDDTest+.txt`

The downloader first tries the raw GitHub path, then falls back to the GitHub API if the raw content host is blocked.

10 Conclusion

The pipeline is now operational and verified on this machine. The main engineering change was making dataset acquisition resilient to host-specific DNS blocking of `raw.githubusercontent.com`. After that correction, the full IDS workflow executed successfully and generated model metrics, explainability outputs, stability measurements, and adversarial-analysis results without further runtime failures.